

Reinforcement Learning on Gym's MiniGrid Interface

Georgios Lazaridis

AEM: 4419



Aristotle University of Thessaloniki

May 2026

1 Introduction and Implementation

This project explores the application of Deep Reinforcement Learning using the **Proximal Policy Optimization (PPO)** algorithm within the *Gym MiniGrid* framework. The core objective was to develop an agent capable of navigating complex, logic-driven environments through custom reward engineering and optimized neural architectures.

1.1 Task Definition and Environment Components

- **Navigation Logic:** The agent must solve multi-stage puzzles requiring inventory management (key collection) and environment manipulation (unlocking doors). Three custom map classes (**First**, **Second**, and **Third**) were developed for this purpose.
- **Custom Entities:** To enhance environmental complexity, the `RimWorldEnemy` class was implemented.
- **Observation Space:** Observations consist of a partially observable 11×11 grid. This spatial data is processed as an image-like tensor, allowing the agent to perceive both static terrain and dynamic threats.

1.2 Architectural Refinements and Evaluation

- **Feature Extraction:** A custom **Convolutional Neural Network (CNN)** serves as the feature extractor. A lightweight visual encoder was implemented, consisting of three consecutive 2D convolutional layers (using 2×2 kernels). These layers progressively expand the feature channels ($16 \rightarrow 32 \rightarrow 64$) before applying a ReLU activation and projecting the spatial state into a 128-dimensional vector through a linear layer.
- **Learning Dynamics:** The **Proximal Policy Optimization (PPO)** algorithm was selected for its robust clipping mechanism, which prevents catastrophic policy updates during the critical phases of reward discovery. The custom CNN was integrated into PPO's `CnnPolicy` actor-critic framework. The network weights were optimized using a learning rate of 3×10^{-4} .
- **Visual Evolution Monitoring:** To assess the agent's progress, `video_overlay.py` and `plot_run_data.py` utility was developed. `video_overlay.py` utilizes **OpenCV** to produce blended video comparisons (`.mp4`) between a baseline random agent and the trained PPO

agents, and `plot_run_data.py` visualizes information regarding the *Average Return* and *Episode Length*

- **Performance Evaluation:** The `evaluate_agents.py` script runs the trained agents through multiple deterministic episodes to calculate key performance metrics, namely the overall *Success Rate* and *Average Return*. This provides a robust statistical validation of the agent’s capabilities across the different environment complexities.

2 Environment Layouts and Map Design

To evaluate the agent’s performance across different levels of complexity, three distinct environment classes were developed. Each map introduces unique spatial constraints and logic requirements.

2.1 Map 1: Hazard Avoidance

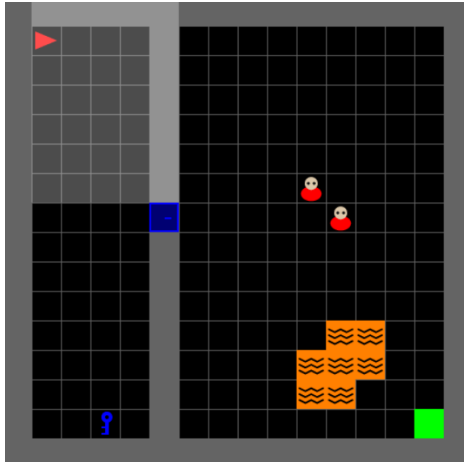


Figure 1: **First Map:** A dual-room configuration featuring a locked door mechanism, lava hazards, and dynamic enemies.

The *First* map (see Figure 1) consists of a two-room layout separated by a locked door. The agent must navigate a simple corridor, avoid lava pits and Enemy entities to reach the goal (indicated by the green square). To address size of the 1616 state space, a dense reward shaping strategy was implemented, utilizing multi-stage sub-goals to effectively guide the agent’s policy convergence.

2.2 Map 2: Sequential Logic and Inventory Management

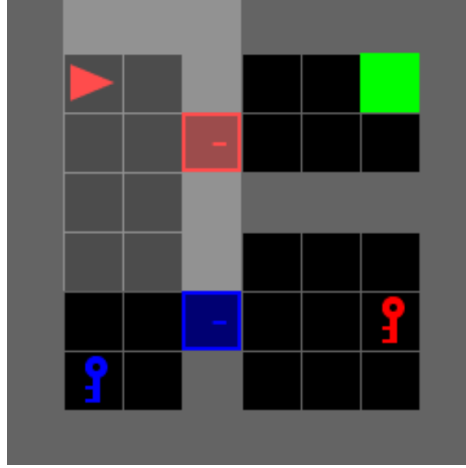


Figure 2: **Second Map:** A multi-stage logic environment requiring key exchange and non-linear pathfinding.

The *Second* map (Figure 2) introduces a different design. It requires multi-stage logic where the agent must pick up a blue key, open the corresponding door, and subsequently drop the first key in order to handle a second key/door sequence before finding the goal. Unlike the *First* map, we follow a sparse reward shaping strategy, given the size of the 8×8 state space.

2.3 Map 3: Exploration and Spatial Generalization

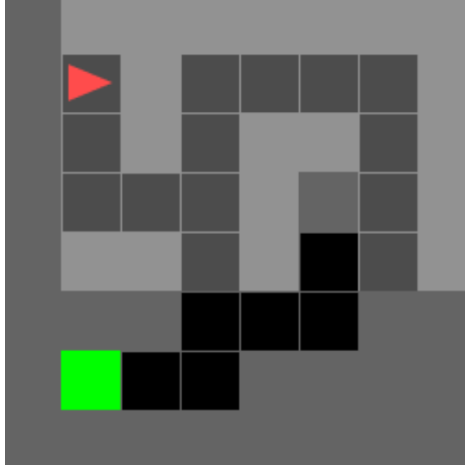


Figure 3: **Third Map:** A dense navigation Maze focused on finding the Shortest Path.

The final *Third* environment (Figure 3) is a dense maze focused on pure navigation efficiency. The maze consists of several dead ends but lacks the enemies and inventory management the other two maps required. Similarly to the *Second* Map, we again follow a sparse reward shaping strategy.

2.4 Custom Entity Rendering

An extension to the environment’s visual capabilities is the implementation of the `RimWorldEnemy` class. The render of the enemy was inspired by the style of games like *RimWorld* and *Prison Architect* to add some flavor.

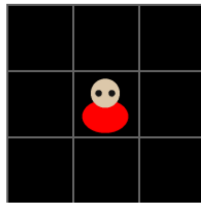


Figure 4: **Custom Enemy Sprite**

The rendering process (see `enemy.py`) utilizes `fill_coords` to layer the body, clothing, and facial features in a specific Z-order, and a helper utility (`point_in_ellipse.py`) to define the boundaries for the color filling.

Similarly to the `Lava` object in *MiniGrid*, any overlap with the `RimWorldEnemy` class triggers an immediate episode termination and a significant reward penalty.

This addition introduces a higher level of visual and logical complexity; the **CNN** model is forced to distinguish between different "lethal" entities (e.g., orange static lava vs. red dynamic enemies), preventing the agent from relying on simple color-blob heuristics.

3 Reward Engineering

3.1 Map 1: Dense Reward Shaping

To accelerate convergence, the task was decomposed into a **Finite State Machine** with sub-goals:

- **Navigation Guidance:** Implemented a "Compass" system using *Manhattan Distance* to provide immediate mathematical feedback during the initial training phases.
- **Sub-goal Rewards:** Assigned discrete bonuses for critical actions (map-specific), +0.5 for key pickup, +1.0 for opening the locked door; small per-step shaping for moving closer to the current phase target (+0.1 when distance decreases in Phases 1–2, and +0.05 when approaching the goal in Phase 3).
- **Time Penalty:** A constant step penalty of -0.005 is applied to incentivize shorter solutions.
- **Failure Penalties:** Collisions with lethal tiles or enemies incur on larger negative rewards and typically terminate the episode.

3.2 Map 2: Sequential Logic and Event-Driven Rewards

The reward structure for the puzzle relies strictly on discrete phase progression and inventory management:

- **Event-Based Sub-goals:** Assigned discrete bonuses for critical actions (map-specific), +0.5 for key pickup, +1.0 for opening the locked door, same as *Map 1*;
- **Inventory Penalty:** To enforce item retention, a strict penalty of -2.0 is applied if the agent drops a required key, which also triggers

immediate episode termination.

3.3 Map 3: Pure Sparse Rewards

In this environment, heuristic guidance and sub-goals are completely eliminated to evaluate the agent’s spatial generalization and exploratory capabilities:

- **Terminal Reward:** The agent receives positive reinforcement solely upon successfully completing the entire mission and reaching the final goal.
- **Time Penalty:** A constant step penalty of -0.05 serves as the only environmental feedback during navigation. This strict sparsity forces the algorithm to rely entirely on intrinsic exploration (entropy) to discover the terminal state without any intermediate breadcrumbs.

4 Results and Evaluation

To assess the convergence the trained PPO agents, the codebase utilizes *TensorBoard* logging for real-time monitoring of `ep_rew_mean` and `ep_len_mean` (stored under `app/data/runs/`). Post-training evaluation was conducted using a stochastic methodology: each model was executed for 500 independent episodes with a strictly enforced truncation limit of 200 steps (`max_steps=200`).

To contextualize the *Average Return*, a Theoretical Maximum Return was calculated for each map, accounting for custom event rewards, distance shaping heuristics, the step-based time penalty (-0.005), and the native MiniGrid completion bonus. The empirical results across 100k, 300k, and 500k timesteps are summarized in Table 1.

Table 1: Evaluation Results across 500 Stochastic Episodes

Environment	Timesteps	Success Rate (%)	Average Return
Map 1 (First)	100k	93.8%	9.585
	300k	100.0%	10.132
	500k	98.4%	9.762
Map 2 (Second)	100k	0.0%	0.911
	300k	0.0%	-0.041
	500k	86.8%	7.343
Map 3 (Third)	100k	100.0%	5.896
	300k	100.0%	5.925
	500k	0.0%	-0.995

4.1 Map 1: Dense Reward Convergence

The theoretical maximum return for Map 1 is approximately 10.35. The evaluation highlights rapid convergence due to the dense reward shaping. The model trained for 300k timesteps achieved a flawless 100% success rate with an average return of 10.132. This indicates near-perfect optimal pathing. The slight variance observed at 100k and 500k timesteps is attributed to stochastic exploratory noise.

Furthermore, after visual inspection of Figure 5 we notice that at approximately 125k timesteps, the 500k agent experienced a severe policy collapse (catastrophic forgetting), with the return plummeting from near-optimal levels to negative values. However, unlike the permanent degradation observed in purely sparse environments, the agent successfully recovered its optimal policy by timestep 250k. The distance-based rewards provided the necessary gradient signals to guide the model out of the collapsed state and stabilize its navigation strategy.

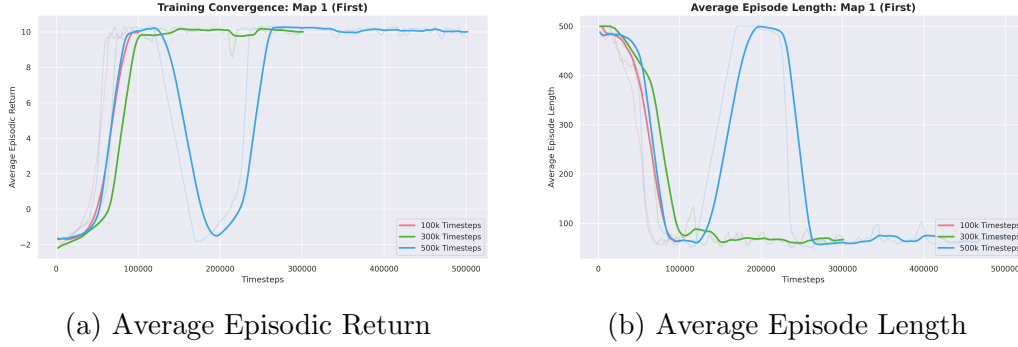


Figure 5: Training metrics for Map 1.

4.2 Map 2: Sequential Logic and Local Minima

Map 2, demonstrated the non-linear progression of deep reinforcement learning. The theoretical maximum for this environment is 8.816. The 100k and 300k models completely failed to reach the terminal state (0% success rate), acquiring only the initial key before becoming trapped in a local minimum and timing out. Extended training allowed the 500k model to escape this local minimum, successfully executing the full sequence to achieve an 86.8% success rate and an average return of 7.343.

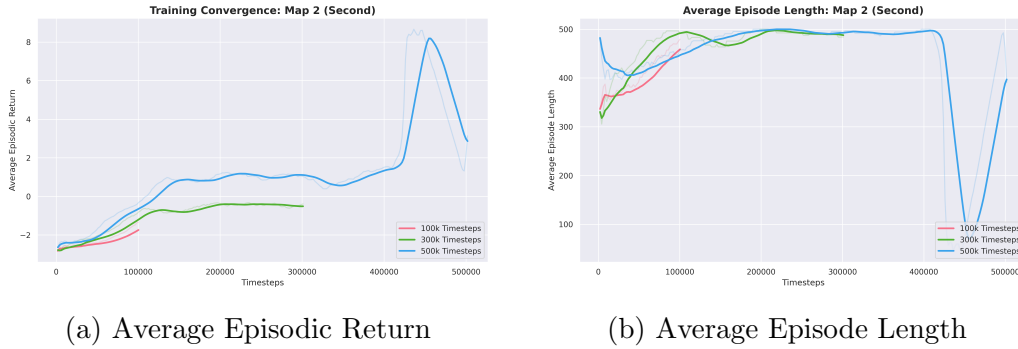


Figure 6: Training metrics for Map 2.

4.3 Map 3: Spatial Exploitation and Catastrophic Forgetting

The evaluation of Map 3 yielded interesting insights regarding agent behavior. The theoretical maximum return for navigating the intended labyrinth was calculated at approximately 5.962. The 300k agent achieved 100% success rate and an average return of 5.925 over 500 episodes.

Interestingly, pushing the training to 500k timesteps negatively impacted the model, as seen in as seen in Figure 7, leading to what appears to be catastrophic forgetting, which was also observed in the first map. However, while the 500k agent in the first map successfully recovered due to its dense rewards, the Map 3 agent’s performance permanently fell to a 0% success rate and a -0.995 average return and with a lack of a dense reward policy was unable to re-converge

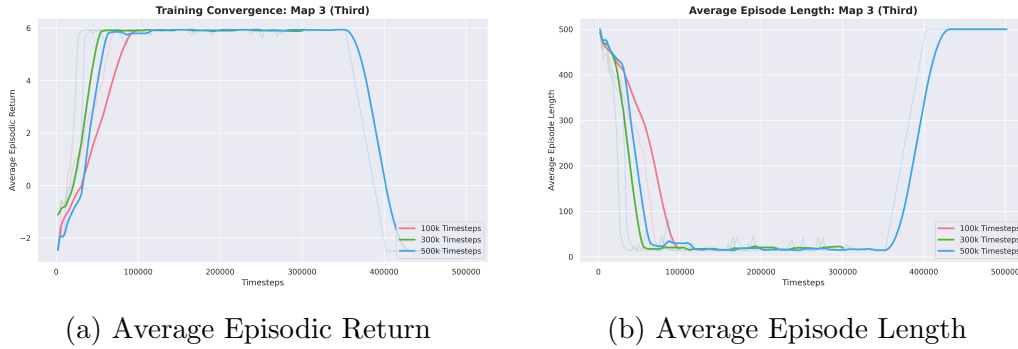


Figure 7: Training metrics for Map 3.

5 Code Implementation and Repository

- **Efficiency:** The codebase utilizes `load` and `save` logic for agents for repeated deployment and testing between agents through the `video_overlay.py` script
- **Saved Models:** Trained agents are saved at `app/data/model/`.

6 GitHub Repository

The complete source code and necessary configuration files for reproducing the experiments are publicly available on the GitHub repository:

- <https://github.com/lazoulios/gym-minigrid>
- **Contents:** This repository includes the main `train.py` script, custom map scripts `first_map.py`, `second_map.py` and `third_map.py` the `enemy.py`, `point_in_ellipse.py` and `video_overlay.py` files, the `requirements.txt` dependency list, and finally `README.md` setup instructions and overall structure report.